

Using Mikroduino SDK USART

Mikroduino USART Library



Mikroduino

Amer Iqbal Qureshi | Mikrotrincs Pakistan

The MikroDuino USART Library

"A good library should allow you to think about your application rather than the hardware. It should simplify common tasks without hiding the power of the underlying microcontroller."

Until now, we have explored serial communication from the ground up. We learned how UART communication works, how a serial frame is constructed, and how the AVR USART hardware sends and receives data. We also studied the USART registers—UCSR0A, UCSR0B, UCSR0C, UBRR0, and UDR0—and learned how each bit controls a specific aspect of communication.

By this point, you know enough to write serial communication programs entirely by manipulating AVR registers. In fact, many experienced embedded programmers still do exactly that.

However, there is a problem.

Programming directly with registers quickly becomes repetitive, difficult to read, and error-prone. Imagine that you simply want to print the word **Hello** to a computer. Before you can transmit even a single character, you must:

- Calculate the baud rate divisor.
- Load the UBRR register.
- Configure the frame format.
- Enable the transmitter.
- Wait until the transmit buffer becomes empty.
- Write the character into UDR0.
- Repeat the waiting process for every character.

Although none of these steps is difficult individually, writing them repeatedly in every project soon becomes tedious.

Consider the following code.

```
UBRR0 = 103;           // 9600 baud @ 16 MHz
UCSR0B = (1 << TXEN0); // Enable transmitter
UCSR0C = (1 << UCSZ01) | (1 << UCSZ00); // 8 data bits
while (!(UCSR0A & (1 << UDRE0)));
UDR0 = 'H';
while (!(UCSR0A & (1 << UDRE0)));
UDR0 = 'i';
```

If you already understand the AVR USART registers, this code is perfectly clear. However, imagine reading it again after six months. You must once again remember:

- What does **TXEN0** do?
- What does **UDRE0** mean?

- Why are bits **UCSZ01** and **UCSZ00** both set?
- Which register controls parity?
- Which register selects the baud rate?

Even experienced programmers often need to consult the datasheet.

Now compare the same task using the MikroDuino SDK USART library.

```
USART0.begin(9600);
USART0.write("Hi");
```

Both programs produce exactly the same result.

The second version is shorter, easier to read, and immediately explains the programmer's intention.

Instead of thinking about registers, we think about the task:

"Initialize the serial port."

"Send this text."

That is exactly what a good library should do.

Why Was This Library Created?

You may wonder, "Why create another USART library when Arduino already provides `Serial`?"

The answer is simple. The Arduino Serial library was designed to be extremely easy for beginners. It hides nearly all of the AVR hardware details and provides a very small set of functions.

For many hobby projects, this is sufficient. However, professional embedded applications often require much more control. For example, suppose you need to communicate with an industrial controller that uses:

- 7 data bits
- Even parity
- Two stop bits

Arduino's Serial library cannot directly configure these settings in the simple `begin()` function. Similarly, imagine you need to:

- Detect framing errors.
- Detect parity errors.
- Detect data overruns.
- Use transmit-only mode.
- Enable the USART Data Register Empty interrupt.

- Use the TX Complete interrupt.
- Support both ATmega328P and ATmega128 with the same code.

These features exist inside the AVR hardware but are either difficult or impossible to access using Arduino's Serial class. The MikroDuino USART library was designed with a different philosophy. Rather than hiding the hardware, it exposes the full power of the AVR USART while still providing a clean and easy-to-use programming interface.

In other words,

The MikroDuino library simplifies programming without taking control away from the programmer.

Design Goals

When designing the MikroDuino SDK, several goals were kept in mind.

Simplicity

The most common operations should require only one line of code.

Instead of writing several register operations,

```
USART0.begin(9600);
```

should configure the entire USART.

Similarly,

```
USART0.writeLine("Hello");
```

should transmit an entire line of text without manually waiting for every character.

Readability

Good programs are easy to understand. Consider these two examples.

Register Programming

```
while (!(UCSR0A & (1 << RXC0)));  
char c = UDR0;
```

MikroDuino SDK

```
char c = USART0.read();
```

The second version almost reads like an English sentence.

When someone else reads your program—or when you return to it after several months—you immediately understand what it does.

Full Hardware Access

Unlike many simplified libraries, the MikroDuino USART library does **not** hide important hardware features.

Nearly every feature offered by the AVR USART module is available through a clean function or configuration option.

For example, the library supports:

- Multiple baud rates
- 5-, 6-, 7-, 8-, and 9-bit characters
- Even and odd parity
- One or two stop bits
- Transmit-only mode
- Receive-only mode
- Full-duplex communication
- Double-speed mode
- Error detection
- Interrupt-driven communication
- Multiple USART peripherals on larger AVR devices

You still have complete control over the hardware—but without constantly manipulating registers.

Zero-Cost Abstraction

One criticism often made about high-level libraries is that they are slower than direct register programming. The MikroDuino USART library was written with embedded systems in mind.

Most functions are very small and compile into only a few machine instructions.

For example,

```
USART0.write('A');
```

still performs exactly the same hardware operation as register programming:

1. Wait until **UDRE0** becomes set.
2. Write the byte into **UDR0**.

The library simply performs these steps for you.

Modern C++ compilers are extremely good at optimizing small functions, so the generated machine code is often almost identical to carefully written register-level code.

As a result, you gain readability and maintainability without sacrificing performance.

The Library Architecture

The MikroDuino SDK treats each USART peripheral as an object.

Instead of writing directly to hardware registers, your program communicates with an object representing that hardware.

For the ATmega328P, the object is:

```
USART0
```

On larger microcontrollers such as the ATmega64 and ATmega128, additional USART peripherals are available.

```
USART0  
USART1
```

Each object controls its own hardware independently.

This means your program can communicate with two completely different serial devices simultaneously.

For example:

- USART0 may be connected to a PC for debugging.
- USART1 may communicate with a GPS receiver.
- Both operate independently with different baud rates and frame formats.

The programmer uses exactly the same functions for both peripherals, making the code portable across multiple AVR microcontrollers.

Your First Impression

As you continue through this chapter, you will notice something interesting. Although the library provides many advanced capabilities—including interrupts, error detection, multiple frame formats, and PROGMEM support—you rarely need to think about hardware registers while writing application code.

Instead, you simply express **what** you want to do:

- Initialize the serial port.
- Send text.

- Receive data.
- Detect errors.
- Enable interrupts.

The library takes care of the low-level details while still allowing experienced programmers to access the full capabilities of the AVR USART hardware whenever required.

Your First USART Program

Every programming library should be easy to use. The MikroDuino USART library was designed with exactly this goal in mind. In most applications, getting the USART running requires only a single function call.

Before writing any code, include the library header.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;
```

The header file contains everything needed to access the USART peripheral. The `using namespace` statement allows us to use the library classes without repeatedly writing the namespace name.

For the ATmega328P, the library automatically creates a global object named **USART0**. This object represents the hardware USART0 peripheral built into the microcontroller.

Rather than writing directly to registers such as `UCSR0A`, `UCSR0B`, `UCSR0C`, and `UDR0`, your program simply communicates with this object.

Think of `USART0` as a software representation of the hardware USART module.

Initializing the USART

The first step in every serial communication program is initialization. Initialization configures all of the hardware required for serial communication, including:

- Baud rate
- Character size
- Parity
- Stop bits
- Transmitter
- Receiver

For the most common configuration—9600 baud, 8 data bits, no parity, one stop bit—the library provides the simplest possible interface.

```
USART0.begin(9600);
```

That single line performs all of the following operations internally:

- Calculates the correct UBRR value.
- Programs the baud rate registers.
- Selects asynchronous mode.
- Configures 8-bit characters.
- Disables parity.
- Selects one stop bit.
- Enables both transmitter and receiver.
- Leaves the USART ready for communication.

Instead of manipulating several hardware registers, the programmer only specifies one parameter—the desired baud rate.

What Happens Internally?

Although the library hides the hardware details, it is useful to understand what happens behind the scenes.

When the following function is called

```
USART0.begin(9600);
```

the library performs approximately these register operations.

```
UBRR0 = 103;
UCSR0A = 0;
UCSR0B = (1<<RXEN0) | (1<<TXEN0);
UCSR0C =
    (1<<UCSZ01) |
    (1<<UCSZ00);
```

You do **not** need to write this code yourself.

The library performs these operations automatically every time `begin()` is called.

This is an excellent example of hardware abstraction. The programmer works with meaningful functions while the library handles the low-level register configuration.

The Default Configuration

Calling

```
USART0.begin(9600);
```

uses the library's default communication settings.

Setting	Default Value
Baud Rate	9600 bps
Data Bits	8
Parity	None
Stop Bits	1
Mode	Transmit + Receive
Double Speed	Disabled

This communication format is commonly written as

```
9600-8-N-1
```

which is supported by almost every serial device. Later in this chapter we will learn how to customize every one of these parameters.

Sending Your First Character

Once the USART has been initialized, transmitting data becomes extremely simple. The smallest piece of information that can be transmitted is a single byte.

The library provides the `write()` function for this purpose.

```
USART0.write('A');
```

This sends the ASCII character **A** over the serial port.

Internally, the library waits until the transmit buffer becomes available before placing the character into the USART data register.

Conceptually, the operation is equivalent to

```
Wait until UDRE0 = 1
↓
Write 'A' into UDR0
↓
Hardware transmits the byte
```

The programmer never needs to check the **UDRE0** flag manually.

Sending Multiple Characters

Suppose we want to send the word

```
Hello
```

Instead of transmitting each character individually,

```
USART0.write('H');
USART0.write('e');
```

```
USART0.write('l');  
USART0.write('l');  
USART0.write('o');
```

we can simply transmit an entire string.

```
USART0.write("Hello");
```

The library automatically sends one character after another until it reaches the end of the string.

Internally, it performs something similar to

```
'H'  
↓  
'e'  
↓  
'l'  
↓  
'l'  
↓  
'o'  
↓  
Null terminator reached  
↓  
Transmission complete
```

This makes programs considerably shorter and easier to understand.

Sending a Complete Line

Most terminal programs display text line by line. Moving to the next line requires sending two special characters:

- Carriage Return (`\r`)
- Line Feed (`\n`)

Together they are written as

```
\r\n
```

Instead of sending these characters manually every time,

```
USART0.write("Temperature = 25");  
USART0.write('\r');  
USART0.write('\n');
```

the library provides a much more convenient function.

```
USART0.writeLine("Temperature = 25");
```

This function automatically appends

```
\r\n
```

after the text.

The output displayed on the terminal becomes

```
Temperature = 25
```

with the cursor positioned at the beginning of the next line. This seemingly small feature saves a surprising amount of repetitive code in larger programs.

Our First Complete Program

Let us combine everything we have learned into a simple program.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

int main()
{
    USART0.begin(9600);

    while (true)
    {
        USART0.writeLine("Hello, MikroDuino!");
    }
}
```

When this program runs, the microcontroller continuously transmits

```
Hello, MikroDuino!
Hello, MikroDuino!
Hello, MikroDuino!
Hello, MikroDuino!
```

to the serial terminal.

If you connect the ATmega328P to a USB-to-Serial converter (or use the on-board USB interface of a MikroDuino/Arduino-compatible board), open the Serial Monitor and set the baud rate to **9600**. You should immediately see the text scrolling continuously across the screen.

Understanding Every Line

Although this program is very short, it demonstrates several important concepts.

```
USART0.begin(9600);
```

Initializes the USART hardware using the default communication format of **9600-8-N-1**.

```
while (true)
```

Creates an infinite loop. Embedded systems rarely terminate like desktop applications. Instead, they execute continuously for as long as power is applied.

```
USART0.writeLine("Hello, MikroDuino!");
```

Transmits the string followed by a carriage return and line feed, causing the terminal to display each message on a new line.

How Much Simpler Is It?

Compare the two approaches.

Register Programming

```
UBRR0 = 103;

UCSR0B = (1<<RXEN0) | (1<<TXEN0);

UCSR0C = (1<<UCSZ01) | (1<<UCSZ00);

while (!(UCSR0A & (1<<UDRE0)));
UDR0 = 'H';
```

MikroDuino Library

```
USART0.begin(9600);

USART0.write('H');
```

Both programs accomplish exactly the same task. However, the second version is easier to read, easier to maintain, and far less prone to programming mistakes. More importantly, it clearly expresses the programmer's intent instead of exposing low-level hardware details.

Configuring the USART

In the previous section, we initialized the USART using a single line of code.

```
USART0.begin(9600);
```

For many applications, this is all you will ever need.

Whether you are communicating with a PC, a Bluetooth module, an ESP8266, or another microcontroller, the default communication format of **9600-8-N-1** works perfectly in most situations.

However, not all serial devices use the same communication settings. Some industrial controllers require:

- 19200 baud
- Even parity
- Two stop bits

Some GPS receivers operate at 4800 baud.

Some laboratory instruments communicate using seven data bits instead of eight.

Some embedded systems require double-speed mode to achieve more accurate baud rates.

If the library only allowed

```
USART0.begin(9600);
```

it would be far too limited.

Fortunately, the MikroDuino USART library allows every important communication parameter to be customized.

The USART Config Structure

Rather than creating several overloaded `begin()` functions with many parameters, the library groups all communication settings into a single configuration object called **USARTConfig**.

Conceptually, you can think of it as a checklist.

```
USARTConfig
✓ Baud Rate
✓ Data Bits
✓ Parity
✓ Stop Bits
✓ Mode
✓ Double Speed
```

You first fill in the options you want, and then pass the completed configuration to the library.

```
USARTConfig cfg;
cfg.baudRate = 115200;
USART0.begin(cfg);
```

This approach has several advantages. First, the code is much easier to read.

Compare these two examples. Imagine a function written like this:

```
USART0.begin(
    115200,
    8,
    NONE,
```

```

    ONE,
    TX_RX,
    false
);

```

Unless you already know the exact order of every parameter, it is difficult to understand.

Now compare it with

```

USARTConfig cfg;

cfg.baudRate = 115200;

cfg.dataBits = USARTDataBits::Eight;

cfg.parity = USARTParity::None;

cfg.stopBits = USARTStopBits::One;

USART0.begin(cfg);

```

Each line clearly explains its purpose. Even someone reading the program months later immediately understands how the USART has been configured.

Default Values

One of the nicest features of `USARTConfig` is that every member already contains a sensible default value.

Internally, the structure looks like this.

```

struct USARTConfig
{
    uint32_t baudRate = 9600;

    USARTDataBits dataBits = USARTDataBits::Eight;

    USARTParity parity = USARTParity::None;

    USARTStopBits stopBits = USARTStopBits::One;

    USARTMode mode = USARTMode::TX_RX;

    bool doubleSpeed = false;
};

```

This means you only need to change the settings that are different from the defaults.

For example, if you only want to increase the baud rate,

```

USARTConfig cfg;

cfg.baudRate = 115200;

```

```
USART0.begin(cfg);
```

Every other parameter remains unchanged.

Internally, the library still uses

- 8 data bits
- No parity
- One stop bit
- Transmit and receive enabled
- Normal speed mode

This makes programs shorter while avoiding unnecessary assignments.

Configuring the Baud Rate

The baud rate determines how fast data is transmitted. It is measured in **bits per second (bps)**.

Some common baud rates are shown below.

Baud Rate	Typical Applications
1200	Legacy equipment
2400	Old modems
4800	GPS receivers
9600	General-purpose communication
19200	Industrial equipment
38400	Embedded systems
57600	Fast communication
115200	Debug consoles and high-speed data

Changing the baud rate is as simple as assigning a new value.

```
USARTConfig cfg;  
cfg.baudRate = 115200;  
USART0.begin(cfg);
```

Internally, the library automatically calculates the correct value for the **UBRR** register. You never need to perform the baud-rate calculations yourself.

Example: High-Speed Communication

Suppose your PC communicates at 115200 baud.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

int main()
{
    USARTConfig cfg;

    cfg.baudRate = 115200;

    USART0.begin(cfg);

    USART0.writeLine("USART initialized.");

    while (true)
    {
    }
}
```

Notice that the code contains no reference to

- UBRR
- UCSR0A
- UCSR0B
- UCSR0C

The library calculates everything automatically.

Configuring the Character Size

Although most serial communication uses eight data bits, the AVR USART hardware actually supports

- 5 bits
- 6 bits
- 7 bits
- 8 bits
- 9 bits

The library represents these using an enumeration.

```
USARTDataBits::Five
USARTDataBits::Six
USARTDataBits::Seven
USARTDataBits::Eight
USARTDataBits::Nine
```


Changing the character size is straightforward.

```
USARTConfig cfg;  
  
cfg.dataBits = USARTDataBits::Seven;  
  
USART0.begin(cfg);
```

The library automatically programs the correct combination of

- UCSZ02
- UCSZ01
- UCSZ00

inside the USART registers. The programmer never needs to remember which bits belong to which register.

When Are Different Character Sizes Used?

Most modern systems use **8-bit characters**.

However, other formats still exist.

Data Bits	Typical Use
5	Historical teleprinters
6	Specialized legacy equipment
7	ASCII terminals, laboratory instruments
8	Modern serial communication
9	Multi-processor communication

For almost every MikroDuino project, you should select

```
USARTDataBits::Eight
```

unless another device specifically requires a different setting.

Configuring Parity

Parity provides a simple method for detecting transmission errors. The AVR USART supports three parity modes.

```
USARTParity::None  
  
USARTParity::Even  
  
USARTParity::Odd
```

To enable even parity,

```
USARTConfig cfg;
```

```
cfg.parity = USARTParity::Even;  
USART0.begin(cfg);
```

To enable odd parity,

```
USARTConfig cfg;  
cfg.parity = USARTParity::Odd;  
USART0.begin(cfg);
```

If parity is not required,

```
cfg.parity = USARTParity::None;
```

or simply leave the default value unchanged.

When Should Parity Be Used?

Parity is useful when communication occurs over long cables or electrically noisy environments.

For example,

- Industrial automation
- Factory machines
- Medical instruments
- Laboratory equipment

For communication with a PC over USB, parity is rarely necessary because USB itself already provides robust error detection.

Configuring Stop Bits

Every serial frame ends with one or more stop bits.

The library provides two options.

```
USARTStopBits::One  
USARTStopBits::Two
```

Example:

```
USARTConfig cfg;  
cfg.stopBits = USARTStopBits::Two;  
USART0.begin(cfg);
```

Internally, this simply sets the **USBS0** bit in **UCSR0C**.

One stop bit is sufficient for almost every application.

Two stop bits are mainly used when communicating with older equipment or slower receivers.

Note: The MikroDuino USART library provides two overloaded versions of `begin()`: a simple version that accepts only the baud rate for the common 9600-8-N-1 configuration, and an advanced version that accepts a `USARTConfig` structure for complete control over all USART parameters. This design keeps simple programs concise while allowing advanced applications to configure every feature of the AVR USART hardware.

Configuring the USART Operating Mode

Until now, every example has initialized the USART using its default operating mode:

```
USART0.begin(9600);
```

or

```
USARTConfig cfg;  
cfg.baudRate = 9600;  
USART0.begin(cfg);
```

In both cases, the library enables **both the transmitter and the receiver**. This is known as **full-duplex communication**, where the microcontroller can transmit and receive data simultaneously.

However, not every application requires both functions. Sometimes the microcontroller only needs to send information. In other applications, it only listens to another device and never transmits.

To support these situations, the MikroDuino USART library provides three operating modes.

Mode	Description
<code>USARTMode::TX</code>	Enable only the transmitter.
<code>USARTMode::RX</code>	Enable only the receiver.
<code>USARTMode::TX_RX</code>	Enable both transmitter and receiver (default).

Internally, these modes simply control the **TXEN0** and **RXEN0** bits in the **UCSR0B** register.

Mode	TXEN0	RXEN0
TX	✓	✗

Mode	TXEN0	RXEN0
RX	X	✓
TX_RX	✓	✓

The library performs these register operations automatically, allowing the programmer to focus on the application's requirements rather than individual control bits.

Transmit-Only Mode

Some embedded systems never receive data.

For example, consider a temperature logger that periodically sends measurements to a PC. The computer simply records the data and never sends commands back to the microcontroller.

Since the receiver is never used, it can remain disabled.

```
USARTConfig cfg;
cfg.mode = USARTMode::TX;
USART0.begin(cfg);
```

In this configuration,

- the transmitter is enabled,
- the receiver is disabled,
- the TX pin operates as a USART output,
- the RX pin is released for normal GPIO use.

Disabling unused hardware can reduce power consumption and frees valuable I/O pins for other purposes.

Receive-Only Mode

Some applications only listen for incoming data. Imagine a microcontroller connected to a GPS receiver. The GPS continuously sends position information, but the microcontroller never needs to transmit anything back.

In this situation, receive-only mode is appropriate.

```
USARTConfig cfg;
cfg.mode = USARTMode::RX;
USART0.begin(cfg);
```

Now,

- the receiver is enabled,

- the transmitter is disabled,
- the RX pin functions as the USART input,
- the TX pin becomes available as a general-purpose I/O pin.

Full-Duplex Mode

The majority of applications require both transmission and reception.

Examples include:

- communicating with a personal computer,
- exchanging data between two microcontrollers,
- Bluetooth modules,
- Wi-Fi modules,
- command-line interfaces,
- interactive serial terminals.

For these applications, use

```
USARTConfig cfg;

cfg.mode = USARTMode::TX_RX;

USART0.begin(cfg);
```

Since this is also the library's default mode, the previous examples

```
USART0.begin(9600);
```

and

```
USARTConfig cfg;

cfg.baudRate = 9600;

USART0.begin(cfg);
```

already operate in full-duplex mode.

Choosing the Correct Mode

Choosing the correct operating mode makes your program more efficient.

Application	Recommended Mode
Debug messages to a PC	TX
GPS receiver	RX
Serial terminal	TX_RX
Bluetooth communication	TX_RX

Application	Recommended Mode
Sensor data logger	TX
Industrial controller	TX_RX

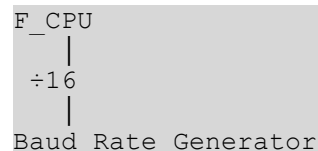
As a general rule, Enable only the hardware that your application actually requires. Doing so simplifies the system, conserves power, and leaves unused pins available for other purposes.

Double-Speed Mode

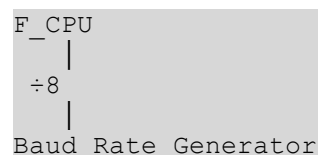
Another important configuration option is **double-speed mode**.

Earlier in this book, we learned that the USART baud-rate generator divides the system clock to produce the serial communication speed.

Normally, the AVR USART divides the CPU clock by **16**.



When double-speed mode is enabled, the divisor changes from **16** to **8**.



This mode is controlled by the **U2X0** bit in the **UCSR0A** register.

Fortunately, the library handles this automatically.

To enable double-speed mode,

```

USARTConfig cfg;

cfg.doubleSpeed = true;

USART0.begin(cfg);

```

No register manipulation is required.

Why Use Double-Speed Mode?

Many beginners assume that double-speed mode simply makes communication twice as fast. This is **not entirely true**.

The actual communication speed is still determined by the baud rate that you specify.

For example,

```
cfg.baudRate = 115200;
```

always requests **115200 baud**, regardless of whether double-speed mode is enabled.

The difference lies in **how the AVR hardware generates that baud rate**.

Using a different clock divisor often produces a baud rate that is much closer to the desired value. This reduces communication errors.

Improving Baud Rate Accuracy

Consider an ATmega328P running at **16 MHz**.

At **115200 baud**, normal mode produces approximately

```
111111 baud
```

which corresponds to an error of about **-3.5%**.

When double-speed mode is enabled, the generated baud rate becomes approximately

```
117647 baud
```

which corresponds to an error of about **+2.1%**.

Although neither value is perfect, the double-speed configuration is significantly closer to the desired baud rate and is generally more reliable when communicating with computers and USB-to-Serial adapters.

For this reason, many experienced AVR programmers enable double-speed mode whenever operating at higher baud rates.

When Should Double-Speed Mode Be Used?

For common baud rates such as

- 2400
- 4800
- 9600
- 19200
- 38400

normal mode already provides excellent accuracy. Double-speed mode becomes most useful when operating at higher baud rates, particularly

- 57600
- 115200

or whenever the baud-rate error table indicates improved accuracy.

Fortunately, the library makes switching between the two modes extremely simple. Simply change one configuration value.

```
USARTConfig cfg;

cfg.baudRate = 115200;

cfg.doubleSpeed = true;

USART0.begin(cfg);
```

Bringing Everything Together

By now, you have learned every configuration option provided by the `USARTConfig` structure. These options can be combined to configure the USART exactly as required.

The following example configures the USART for communication with an industrial device operating at **115200 baud, 8 data bits, even parity, one stop bit, full-duplex mode, and double-speed operation**.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

int main()
{
    USARTConfig cfg;

    cfg.baudRate      = 115200;
    cfg.dataBits      = USARTDataBits::Eight;
    cfg.parity        = USARTParity::Even;
    cfg.stopBits      = USARTStopBits::One;
    cfg.mode          = USARTMode::TX_RX;
    cfg.doubleSpeed   = true;

    USART0.begin(cfg);

    while (true)
    {

    }
}
```

Although this configuration appears much more detailed than the earlier examples, it remains remarkably easy to read. Every communication parameter is explicitly named, making the program almost self-documenting.

Transmitting Data

Configuring the USART is only the first step. Once the hardware has been initialized, the real purpose of serial communication begins—**sending information**.

Almost every embedded application needs to transmit data at some point. For example, a program may need to:

- Display a startup message.
- Report sensor readings.
- Send commands to another microcontroller.
- Print debugging information.
- Communicate with a GPS receiver, Bluetooth module, or Wi-Fi module.
- Transfer binary data between embedded systems.

The MikroDuino USART library provides a collection of functions that make transmitting data both simple and efficient.

In this section, we will explore these functions one by one, beginning with the smallest unit of information—a single byte.

Sending a Single Character

The simplest transmission function is

```
write(uint8_t data)
```

It sends exactly one byte through the USART.

The function prototype is

```
void write(uint8_t data);
```

where

Parameter	Description
data	The byte to be transmitted.

The function does not return a value.

Sending an ASCII Character

Characters are enclosed in single quotation marks.

```
USART0.write('A');
```

This sends the ASCII code for the letter **A**.

Likewise,

```
USART0.write('B');
```

```
USART0.write('C');
```

```
USART0.write('D');
```

transmits

```
ABCD
```

one character at a time.

Sending Raw Binary Data

Remember that the USART transmits **bytes**, not just printable characters.

Therefore, the following code is equally valid.

```
USART0.write(0x55);  
USART0.write(0xAA);  
USART0.write(0xFF);  
USART0.write(0x00);
```

These hexadecimal values may represent

- sensor packets,
- communication protocols,
- device commands,
- binary files

or any other form of digital information.

Unlike strings, `write()` does not interpret the data in any way. Whatever byte you provide is transmitted exactly as it is.

What Happens Internally?

Although the library hides the hardware details, it is useful to understand what occurs inside the microcontroller when `write()` is called.

Suppose we execute

```
USART0.write('A');
```

Internally, the library performs approximately the following sequence.

```
Application  
↓  
USART0.write()  
↓  
Wait until UDRE0 becomes 1  
↓  
Load UDR0  
↓  
Hardware Shift Register
```

↓
TX Pin
↓
Serial Cable

The most important step is waiting for **UDRE0** (USART Data Register Empty). This flag indicates that the transmit data register is ready to accept another byte. Only after this flag becomes set does the library write the byte into **UDR0**.

The USART hardware then automatically shifts the bits onto the TX pin while the CPU continues executing the remainder of the program.

Register Programming vs. Library

Without the library, transmitting a single character requires manually checking the hardware status.

```
while (!(UCSR0A & (1 << UDRE0)))  
{  
  
}  
  
UDR0 = 'A';
```

Using the MikroDuino library, the same operation becomes

```
USART0.write('A');
```

Both approaches perform exactly the same hardware operation. The difference is that the library hides the repetitive register manipulation, making the programmer's intention immediately obvious.

Example 1 – Printing the Alphabet

The following program prints the uppercase English alphabet.

```
#include <mikroduino/usart.hpp>  
  
using namespace MikroDuino;  
  
int main()  
{  
    USART0.begin(9600);  
  
    for(char c='A'; c<='Z'; c++)  
    {  
        USART0.write(c);  
    }  
  
    while(true)  
    {  
  
    }  
}
```

The terminal displays

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

Notice that no spaces or line breaks are inserted. Each call to `write()` simply sends the next character.

Example 2 – Sending Numbers as Bytes

Consider the following program.

```
USART0.write(65);  
USART0.write(66);  
USART0.write(67);
```

The terminal displays

```
ABC
```

Why?

Because the values

Decimal	ASCII Character
---------	-----------------

65	A
----	---

66	B
----	---

67	C
----	---

represent printable ASCII codes.

This is an important concept.

The `write()` function sends **bytes**, not decimal numbers.

If you write

```
USART0.write(123);
```

the library transmits the single byte

```
0x7B
```

which corresponds to the ASCII character

```
{
```

It **does not** transmit the three characters

```
123
```

If your goal is to display the decimal number **123**, you should use `writeInt()`, which we will study later in this chapter.

Blocking Behavior

The `write()` function is a **blocking** function. If the USART is still busy transmitting the previous byte, the function waits until the hardware is ready before loading the next byte.

For a single character, this delay is usually so short that it is unnoticeable. However, when transmitting large amounts of data, the CPU repeatedly waits for the hardware to become ready.

For many applications this is perfectly acceptable because serial communication is relatively slow compared to the AVR processor. Nevertheless, some real-time applications cannot afford to wait.

For these situations, the library provides more advanced techniques, including:

- `txReady()` for non-blocking transmission,
- the **UDRE interrupt** for interrupt-driven transmission,
- asynchronous transmission using interrupt service routines.

These advanced techniques will be covered later in this chapter.

Best Practices

When using `write()`, keep the following recommendations in mind.

Use `write()` for single bytes.

Characters, binary packets, control codes, and protocol bytes are ideal candidates.

Use `write(const char*)` for text strings.

It is much more convenient than sending every character individually.

Use `writeInt()` for decimal numbers.

Do not expect `write(123)` to print the digits "123".

Use `writeHex()` when displaying register values or memory addresses.

It produces formatted hexadecimal output that is much easier to read during debugging.

Sending Text Strings

In the previous section, we learned how to transmit a single byte using the `write()` function.

```
USART0.write('A');
```

Although this works perfectly, transmitting an entire sentence one character at a time would be extremely tedious.

Imagine writing

```
USART0.write('H');  
USART0.write('e');  
USART0.write('l');  
USART0.write('l');  
USART0.write('o');
```

just to display the word **Hello**.

There is clearly a better way.

The MikroDuino USART library provides another version of the `write()` function that accepts a **C-style string**.

```
void write(const char* str);
```

Instead of sending one character, this function sends an entire string.

Sending Your First String

Using the string version of `write()` is very simple.

```
USART0.write("Hello World");
```

The terminal displays

```
Hello World
```

Notice that the quotation marks are **double quotation marks**.

```
"Hello World"
```

This tells the compiler that the data is a **string** rather than a single character.

How Does It Work?

A C string is simply an array of characters stored one after another in memory.

For example,

```
"Hello"
```

is actually stored as

Address Value

0	H
1	e
2	l

Address Value

3	
4	o
5	'\0'

The final byte

```
'\0'
```

is called the **null terminator**.

It marks the end of the string. The `write()` function repeatedly sends characters until it encounters this special byte. Conceptually, it performs something similar to

```
Read first character
↓
Transmit
↓
Move to next character
↓
Transmit
↓
Repeat
↓
Null terminator found
↓
Stop
```

This entire process is handled automatically by the library.

Example – Displaying a Welcome Message

A common practice in embedded systems is to display a startup message immediately after the microcontroller begins executing.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

int main()
{
    USART0.begin(9600);

    USART0.write("MikroDuino SDK");
    USART0.write("\r\n");

    while(true)
    {

    }
}
```

The terminal displays

```
MikroDuino SDK
```

Although this works correctly, the library provides an even more convenient function for printing complete lines.

Writing Complete Lines

Most terminal programs expect each line of text to end with two special characters.

These are

- Carriage Return (`\r`)
- Line Feed (`\n`)

Together they are written as

```
\r\n
```

Rather than transmitting these characters manually every time,

```
USART0.write("System Ready");  
USART0.write("\r\n");
```

the library provides the function

```
USART0.writeLine("System Ready");
```

Internally, this is equivalent to

```
USART0.write("System Ready");  
USART0.write('\r');  
USART0.write('\n');
```

The output appears as

```
System Ready
```

with the cursor automatically moving to the beginning of the next line.

Example – Printing Multiple Lines

```
USART0.writeLine("MikroDuino SDK");  
USART0.writeLine("USART Library");  
USART0.writeLine("Version 1.0");
```

Output

```
MikroDuino SDK  
USART Library  
Version 1.0
```

Programs become much cleaner because you never need to remember to append `\r\n`.

A Hidden Problem: SRAM Usage

At first glance, using

```
USART0.write("Hello World");
```

seems perfectly harmless.

Unfortunately, on AVR microcontrollers there is an important detail that every programmer should understand.

The ATmega328P contains

- **32 KB Flash Memory**
- **2 KB SRAM**

Flash memory is used to store your program. SRAM is used for variables while the program is running.

The problem is that ordinary string literals such as

```
"Hello World"
```

are copied into SRAM during program startup. This means every constant message occupies valuable RAM.

For a single string, this is not a problem. However, consider a program containing dozens of menu items.

```
"Main Menu"  
"Settings"  
"Temperature"  
"Humidity"  
"Motor Running"  
"Communication Error"  
"Configuration Saved"  
...
```

Each one consumes SRAM. Eventually the program may run out of RAM, leading to mysterious crashes, corrupted variables, or complete system failure.

The AVR Solution: PROGMEM

AVR microcontrollers provide a special memory area called **Program Memory (PROGMEM)**.

Instead of copying constant strings into SRAM, they remain stored in Flash memory. This provides two major advantages.

- More SRAM remains available for variables.
- Larger applications become possible.

Unfortunately, reading directly from Flash memory requires special functions. This is one reason many beginners find PROGMEM programming difficult.

Built-in PROGMEM Support

The MikroDuino USART library makes this remarkably easy.

Instead of

```
USART0.write("System Ready");
```

simply write

```
USART0.write_P(PSTR("System Ready"));
```

The text is now transmitted directly from Flash memory.

No SRAM is wasted.

The programmer does not need to manually read characters using `pgm_read_byte()`.

The library performs everything internally.

What Does PSTR() Do?

The macro

```
PSTR("System Ready")
```

instructs the compiler to place the string inside Flash memory instead of SRAM.

The library's `write_P()` function then reads one character at a time directly from Flash.

Conceptually, the process becomes

```
Flash Memory
↓
Read one character
↓
Transmit
↓
Read next character
↓
Transmit
↓
Repeat until null terminator
```

Everything happens automatically.

Example – Memory-Efficient Startup Message

```
#include <avr/pgmspace.h>

USART0.writeLine_P(
    PSTR("MikroDuino USART Library")
);
```

The output is identical to

```
USART0.writeLine("MikroDuino USART Library");
```

The difference is entirely in memory usage.

Function	String Location
<code>write()</code>	SRAM
<code>write_P()</code>	Flash (PROGMEM)

Which Function Should You Use?

For beginners, this is an easy rule to remember.

Data Type	Recommended Function
Single character	<code>write()</code>
Variable string stored in RAM	<code>write()</code>
Constant text	<code>write_P()</code>
Constant text with newline	<code>writeLine_P()</code>

As your programs become larger, using `write_P()` consistently can save hundreds of bytes of SRAM—an enormous amount on an ATmega328P with only 2 KB of RAM.

Best Practice

Whenever you transmit text that never changes during program execution—such as menus, prompts, error messages, or startup banners—prefer `write_P()` or `writeLine_P()`. This keeps the string in Flash memory instead of SRAM, leaving precious RAM available for variables, buffers, and the program stack. Making this a habit from the beginning will help your programs remain efficient and scalable as they grow in complexity.

Transmitting Numbers

So far, we have transmitted

- single characters,
- text strings,

- complete lines of text.

However, embedded systems rarely communicate using text alone. A microcontroller often needs to send numerical information such as:

- Temperature
- Humidity
- Voltage
- Distance
- ADC readings
- Counter values
- Sensor IDs
- Error codes

For example, suppose an analog-to-digital converter measures a voltage and returns

```
512
```

If we simply write

```
USART0.write(512);
```

the terminal will **not** display

```
512
```

Instead, only the lowest eight bits of the value are transmitted as a single byte.

This is because the `write()` function sends **raw binary data**, not decimal numbers.

To display numbers in a human-readable form, the MikroDuino USART library provides several conversion functions.

Writing Decimal Integers

The simplest function is

```
writeInt()
```

Its purpose is to convert an integer into decimal ASCII characters before transmitting it.

The function prototype is

```
void writeInt(int32_t value);
```

Parameter	Description
value	Integer value to be transmitted.

The function converts the number internally and sends each digit one after another.

Example

```
USART0.writeInt(12345);
```

The terminal displays 12345

Internally, the library converts

```
12345
```

into

```
'1'  
↓  
'2'  
↓  
'3'  
↓  
'4'  
↓  
'5'
```

before transmitting the characters.

The programmer never needs to perform this conversion manually.

Positive and Negative Numbers

The function automatically handles signed integers.

```
USART0.writeInt(250);  
USART0.write(' ');  
USART0.writeInt(-75);
```

Output

```
250 -75
```

Notice that the minus sign is generated automatically.

Example – Displaying an ADC Reading

Suppose an ADC conversion returns

```
uint16_t adcValue = 678;
```

Displaying the result is as simple as

```
USART0.write("ADC = ");
```

```
USART0.writeInt(adcValue);  
USART0.writeLine("");
```

Output

```
ADC = 678
```

Without `writeInt()`, the programmer would first have to convert the integer into a character string using functions such as `itoa()` before sending it. The library performs this conversion automatically.

Writing Unsigned Integers

Many embedded applications use unsigned variables because they can represent larger positive values.

For example,

```
uint16_t counter = 65535;
```

The library also supports unsigned integers.

```
USART0.writeUInt(counter);
```

Output

```
65535
```

Unsigned values are especially useful for

- Timer counts
- ADC readings
- Memory addresses
- Packet counters
- Loop counters

Since negative numbers are impossible, the entire numeric range is available for positive values.

Writing Long Numbers

Sometimes an application needs to display numbers larger than 16 bits.

Examples include

- Millisecond timers
- File sizes
- System uptime
- GPS coordinates
- Communication statistics

The library supports larger integer types transparently.

```
uint32_t totalBytes = 123456789;  
USART0.writeUInt(totalBytes);
```

Output

```
123456789
```

The programmer does not need to worry about how many digits are required.

The library automatically converts the entire value.

Combining Text and Numbers

One of the most common tasks in embedded programming is combining descriptive text with measured values.

For example,

```
USART0.write("Temperature = ");  
USART0.writeInt(25);  
USART0.writeLine(" C");
```

Output

```
Temperature = 25 C
```

Another example

```
USART0.write("Speed = ");  
USART0.writeInt(120);  
USART0.writeLine(" km/h");
```

Output

```
Speed = 120 km/h
```

This style of programming produces clean, readable output without requiring complicated formatting functions.

A Practical Example

Consider a simple sensor monitoring application.

```
#include <mikroduino/usart.hpp>  
  
using namespace MikroDuino;  
  
int main()
```

```

{
    USART0.begin(9600);

    int temperature = 24;
    int humidity = 68;

    while(true)
    {
        USART0.write("Temperature : ");
        USART0.writeInt(temperature);
        USART0.writeLine(" C");

        USART0.write("Humidity      : ");
        USART0.writeInt(humidity);
        USART0.writeLine(" %");

        USART0.writeLine("-----");
    }
}

```

The serial terminal displays

```

Temperature : 24 C
Humidity      : 68 %
-----
Temperature : 24 C
Humidity      : 68 %
-----

```

Even though the program repeatedly displays numerical values, no manual number-to-string conversion is required.

Why Not Use `itoa()`?

Experienced C programmers may already know functions such as

- `itoa()`
- `ltoa()`
- `sprintf()`

to convert integers into strings.

For example,

```

char buffer[10];

itoa(value, buffer, 10);

USART0.write(buffer);

```

Although this works, it has several disadvantages.

- An additional character buffer is required.

- More RAM is consumed.
- More code must be written.
- The program becomes less readable.

With the MikroDuino library, the same task becomes

```
USART0.writeInt(value);
```

One function performs the entire operation.

Register Programming vs. Library

Displaying an integer using register programming is surprisingly complicated.

The programmer must

1. Convert the integer into ASCII.
2. Store the result in a character buffer.
3. Transmit every character individually.
4. Wait for the transmit buffer before each byte.

With the MikroDuino USART library, all of these steps are hidden behind a single function call.

This allows you to concentrate on your application instead of repeatedly writing conversion routines.

Best Practice

Whenever you want the user to **read** a number, use

```
USART0.writeInt(value);
```

Whenever another microcontroller or communication protocol expects **raw binary data**, use

```
USART0.write(value);
```

Understanding this distinction is extremely important.

- **write()** sends the numeric value as one or more raw bytes.

- `writeInt()` converts the number into human-readable decimal digits before transmitting it.

Choosing the correct function ensures that your program communicates exactly what you intend.

Writing Hexadecimal Numbers

In embedded programming, decimal numbers are not always the best way to represent data.

Consider the following value.

```
170
```

Although perfectly correct, it tells us very little about the actual bit pattern stored inside the microcontroller.

Now look at the same value in hexadecimal.

```
0xAA
```

Immediately, an experienced programmer recognizes the familiar alternating bit pattern

```
10101010
```

This is one reason why datasheets, memory maps, communication protocols, and hardware registers are almost always described using hexadecimal notation.

The MikroDuino USART library provides the `writeHex()` function to display numbers in hexadecimal format.

The `writeHex()` Function

The function prototype is

```
void writeHex(uint32_t value);
```

Parameter	Description
<code>value</code>	Integer value to be displayed in hexadecimal.

Instead of displaying decimal digits, the function converts the value into hexadecimal characters before transmitting them.

Example

```
USART0.writeHex(255);
```

Output

```
FF
```

Likewise,

```
USART0.writeHex(26);
```

Output

```
1A
```

Notice that hexadecimal uses the digits

```
0 1 2 3 4 5 6 7 8 9 A B C D E F
```

where

Decimal	Hexadecimal
---------	-------------

10	A
11	B
12	C
13	D
14	E
15	F

Why Hexadecimal?

Imagine reading the contents of an AVR register.

```
uint8_t status = 0x85;
```

If we print it as a decimal number,

```
USART0.writeInt(status);
```

the output becomes

```
133
```

This value is mathematically correct, but it is difficult to compare with the AVR datasheet.

Now display the same value using

```
USART0.writeHex(status);
```

Output

```
85
```

This immediately matches the register value shown in the datasheet.

For embedded engineers, hexadecimal notation is usually much more meaningful than decimal.

Displaying Register Values

One of the most common debugging techniques is displaying hardware registers.

For example,

```
USART0.write("UCSR0A = 0x");  
USART0.writeHex(UCSR0A);  
USART0.writeLine("");
```

Possible output

```
UCSR0A = 0x20
```

This allows you to verify

- status flags,
- interrupt flags,
- error conditions,
- communication state,

without using a hardware debugger.

Displaying Sensor Packets

Communication protocols often transmit binary packets.

Suppose a sensor returns

```
uint8_t packet[] =  
{  
    0x55,  
    0xAA,  
    0x10,  
    0x35  
};
```

Displaying the packet becomes very simple.

```
for(uint8_t i=0; i<4; i++)  
{  
    USART0.writeHex(packet[i]);
```

```
    USART0.write(' ');  
}
```

Output

```
55 AA 10 35
```

Notice how much easier this is to read than

```
85 170 16 53
```

which is the same data expressed in decimal.

Example – ADC Debugging

Suppose an ADC conversion returns

```
uint16_t adcValue = 768;
```

You can display both decimal and hexadecimal representations.

```
USART0.write("Decimal : ");  
USART0.writeInt(adcValue);  
USART0.writeLine("");  
USART0.write("Hex      : 0x");  
USART0.writeHex(adcValue);  
USART0.writeLine("");
```

Output

```
Decimal : 768  
Hex      : 0x300
```

Displaying both formats is often useful during debugging.

Example – Memory Address Display

Hexadecimal notation is also the standard way to display memory addresses.

```
uint16_t address = 0x01F4;  
USART0.write("Address = 0x");  
USART0.writeHex(address);  
USART0.writeLine("");
```

Output

```
Address = 0x01F4
```

Nearly every embedded debugger, datasheet, and compiler uses hexadecimal addresses, making this notation familiar to embedded developers.

Decimal vs. Hexadecimal

Consider the following examples.

Function Call	Output
<code>writeInt(255)</code>	255
<code>writeHex(255)</code>	FF
<code>writeInt(16)</code>	16
<code>writeHex(16)</code>	10
<code>writeInt(26)</code>	26
<code>writeHex(26)</code>	1A

Neither format is "better." Each serves a different purpose.

Use **decimal** when the value is intended for people to read, such as temperatures, voltages, distances, or sensor readings.

Use **hexadecimal** when working with hardware, memory, registers, communication packets, or debugging information.

A Practical Debugging Example

The following program displays the contents of the USART status register every second.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

int main()
{
    USART0.begin(9600);

    while(true)
    {
        USART0.write("UCSR0A = 0x");

        USART0.writeHex(UCSR0A);

        USART0.writeLine("");

        // Delay here
    }
}
```

As communication occurs, the displayed hexadecimal value changes to reflect the current status of the USART hardware. This simple technique is invaluable when troubleshooting communication problems.

Register Programming vs. Library

Without the library, displaying a hexadecimal value requires several steps.

1. Convert the number to hexadecimal characters.
2. Store the result in a buffer.
3. Send each character individually.
4. Handle leading zeros if required.

With the MikroDuino library, the entire operation becomes

```
USART0.writeHex(value);
```

The library performs all formatting automatically, allowing you to focus on understanding the data rather than converting it.

Best Practice

A good rule of thumb is:

- Use `writeInt()` for values that users or operators need to read and understand.
- Use `writeHex()` for values that engineers need to debug or compare with datasheets.

Many experienced embedded programmers display both formats during development. For example:

```
Temperature = 25 °C  
ADC Value   = 768 (0x300)  
Status      = 0x24
```

This combination provides a clear human-readable value while also exposing the underlying hardware representation, making debugging faster and more intuitive.

`writeFloat()` – Transmitting Floating-Point Numbers

Many embedded applications work with floating-point values such as temperature, humidity, voltage, pressure, and sensor measurements. The `writeFloat()` function converts a floating-point number into its ASCII representation and transmits it over the USART.

Function Prototype

```
void writeFloat(float value, uint8_t precision = 2);
```

Parameter	Description
value	The floating-point value to transmit.
precision	Number of digits to display after the decimal point. The default is 2.

Example 1 – Temperature

```
float temperature = 25.75;

USART0.write("Temperature = ");
USART0.writeFloat(temperature);
USART0.writeLine(" C");
```

Output

```
Temperature = 25.75 C
```

Example 2 – Specifying Precision

```
float voltage = 4.9876;

USART0.write("Voltage = ");
USART0.writeFloat(voltage, 3);
USART0.writeLine(" V");
```

Output

```
Voltage = 4.988 V
```

Notes

- The default precision is **2 decimal places**.
- Increase the `precision` parameter when more decimal digits are required.
- `writeFloat()` is ideal for displaying sensor values, while `writeInt()` should be used for whole numbers.

Excellent. Up to this point, the reader has learned how to **send** information. The next logical step is to learn how to **receive** it. As before, we should start with the concepts before introducing the API.

Receiving Data

Communication is a two-way process. In the previous sections, we learned how the ATmega328P transmits characters, strings, and numbers to another device. However, many embedded applications also need to **receive** information.

For example, a microcontroller may receive

- Commands from a PC
- Configuration data from a terminal
- GPS coordinates from a GPS receiver
- Sensor measurements from another microcontroller
- Responses from a Bluetooth or Wi-Fi module
- Binary packets from an industrial controller

Without the ability to receive data, the microcontroller can only "talk." To build interactive systems, it must also be able to "listen."

The MikroDuino USART library provides a simple set of functions for receiving data without requiring direct access to the AVR USART registers.

How USART Reception Works

When a serial device sends a character to the ATmega328P, the USART hardware automatically performs the following steps:

1. Detects the Start Bit.
2. Receives each data bit according to the configured baud rate.
3. Checks the parity bit (if enabled).
4. Verifies the stop bit.
5. Stores the received byte in the Receive Buffer (UDR0).
6. Sets the **RXC0** (Receive Complete) flag.

Once the **RXC0** flag is set, the received byte is ready for your program to read. Fortunately, the library manages these hardware details internally.

Checking for Available Data

Before reading a character, the program should first determine whether data has actually been received.

The library provides the function

```
available()
```

Function Prototype

```
bool available();
```

Returns

- `true` if at least one character has been received.
- `false` if no data is available.

Internally, this function simply checks whether the **RXC0** flag has been set by the USART hardware.

Example

```
if (USART0.available())  
{  
    // Data has arrived  
}
```

This simple test prevents the program from attempting to read data that has not yet been received.

Reading a Character

Once data is available, it can be read using

```
read()
```

Function Prototype

```
uint8_t read();
```

The function returns the received byte.

Example:

```
char ch = USART0.read();
```

If the sender transmitted

```
A
```

then

```
ch
```

contains

```
'A'
```

The library automatically retrieves the byte from **UDR0** and clears the receive flag so the USART can accept the next character.

Your First Receiving Program

The following program waits until a character is received and immediately sends it back.

```
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

int main()
{
    USART0.begin(9600);

    while (true)
    {
        if (USART0.available())
        {
            char ch = USART0.read();

            USART0.write(ch);
        }
    }
}
```

This program is called a **serial echo program**.

Whatever the user types into the serial terminal immediately appears back on the screen.

Example

User types

HELLO

Output

HELLO

Although very simple, this example demonstrates the complete cycle of serial communication:

```
PC
↓
Transmit Character
↓
USART Receiver
↓
Receive Buffer
↓
read()
↓
write()
↓
USART Transmitter
↓
PC
```

Blocking vs. Non-Blocking Reception

Notice that the program first calls

```
USART0.available()
```

before calling

```
USART0.read();
```

This approach is called **non-blocking reception** because the CPU continues executing while waiting for data. If no character has arrived, the program simply continues executing the remainder of the loop. This is the recommended approach for most embedded applications.

Reading Multiple Characters

Receiving one character is useful, but most commands consist of several characters.

For example, the user may type

```
LED ON
```

The USART receives these characters one at a time.

```
L
↓
E
↓
D
↓
Space
↓
O
↓
N
```

Each call to

```
USART0.read();
```

returns only the next available character.

Your program is responsible for combining these characters into complete words or commands.

Later in this chapter, we will learn how to receive entire strings and command lines using the higher-level functions provided by the library.

Register Programming vs. Library

Without the MikroDuino library, receiving a character requires directly accessing the USART status register.

```
while (!(UCSR0A & (1 << RXC0)))
{
```

```
}  
  
char ch = UDR0;
```

The program must continuously monitor the **RXC0** flag until a character arrives. Using the MikroDuino library, the same operation becomes

```
if (USART0.available())  
{  
    char ch = USART0.read();  
}
```

The library hides the register manipulation while making the code easier to understand.

Best Practice

Always check whether data is available before attempting to read it.

```
if (USART0.available())  
{  
    char ch = USART0.read();  
}
```

This simple pattern is efficient, easy to understand, and suitable for the vast majority of embedded applications.

Receiving Text Strings

Reading a single character is useful, but real applications usually exchange complete words, commands, or sentences rather than individual bytes. Consider a serial terminal connected to the ATmega328P.

The user may type

```
LED ON
```

Although this appears as a single command on the screen, the USART hardware actually receives one character at a time.

The microcontroller must collect these characters and combine them into a complete string before the command can be interpreted. Doing this manually quickly becomes tedious.

Fortunately, the MikroDuino USART library provides higher-level functions that automatically assemble incoming characters into strings. Instead of worrying about buffers, counters, and null terminators, the programmer simply requests an entire line of text.

Receiving an Entire Line

The most common form of serial communication is line-oriented communication. The user types a command and presses **Enter**. For example,

```
STATUS
```

or

```
LED OFF
```

The library continues receiving characters until the line ending ('`\n`' or "`\r\n`") is detected. At that point, the complete command is returned to your application.

Conceptually, the process looks like this.

```
User Types
↓
USART Receives Characters
↓
Internal Receive Buffer
↓
Newline Received
↓
Return Complete String
```

This approach is much simpler than processing every character individually.

Example – Reading a Command

Suppose the library provides a function similar to

```
readLine(buffer, sizeof(buffer));
```

The program becomes remarkably simple.

```
char command[32];

while (true)
{
    if (USART0.readLine(command, sizeof(command)))
    {
        USART0.write("You entered: ");
        USART0.writeLine(command);
    }
}
```

Example terminal session

```
> STATUS
You entered: STATUS

> HELP
You entered: HELP
```

Notice how the application receives an entire command instead of processing individual characters. This makes command interpreters much easier to write.

Building a Simple Command Interpreter

Receiving complete strings opens the door to interactive applications.

For example,

```
LED ON
LED OFF
STATUS
RESET
HELP
```

can all be processed using ordinary string comparisons.

```
if(strcmp(command, "LED ON") == 0)
{
    ledOn();
}
else if(strcmp(command, "LED OFF") == 0)
{
    ledOff();
}
else if(strcmp(command, "STATUS") == 0)
{
    printStatus();
}
```

Notice that the serial communication code disappears almost completely. The program simply receives a string and decides what to do with it. This is exactly how many embedded command-line interfaces (CLI) are implemented.

Buffer Size Considerations

Every received string must be stored somewhere in memory. This storage area is called a **receive buffer**.

For example,

```
char command[32];
```

can hold

- up to 31 printable characters
- plus the terminating null character.

If the user enters more characters than the buffer can hold, the excess characters must either be discarded or handled appropriately by the application. For this reason, always choose a buffer large enough for the expected commands.

For menu-driven applications, 32 or 64 bytes is usually sufficient.

Receiving Binary Data

Not every USART application transmits text. Many embedded systems exchange binary packets.

For example,

```
0x55 0xAA 0x01 0x03 0x2F
```

In these situations, reading complete text lines is inappropriate because binary packets may contain values that are not printable characters.

Instead, read each byte individually using

```
uint8_t value = USART0.read();
```

and process the packet according to the communication protocol.

Choosing the Right Receive Method

The MikroDuino USART library offers different receive techniques depending on the application.

Data Type	Recommended Method
Single key press	<code>read()</code>
Individual bytes	<code>read()</code>
Terminal commands	<code>readLine()</code>
Text messages	<code>readString()</code>
Binary packets	<code>read()</code>
Interactive console	<code>readLine()</code>

Selecting the appropriate function keeps your code simple while avoiding unnecessary parsing logic.

Interrupt-Driven Communication

Until now, every example in this chapter has used **polling**. The program repeatedly checks

```
USART0.available()
```

to determine whether data has arrived. Although this method is easy to understand, it has one disadvantage. The CPU must continuously ask the USART "Has a character arrived yet?"

Most of the time the answer is "No."

This repeated checking wastes processing time.

The AVR USART provides a much more efficient solution called **interrupt-driven communication**.

Instead of constantly polling the USART, the CPU continues executing the main program. Whenever a character arrives, the USART automatically interrupts the CPU and requests immediate attention.

The sequence is shown below.

```
Main Program Running
↓
Character Arrives
↓
USART Sets RXC0
↓
USART Interrupt
↓
Interrupt Service Routine (ISR)
↓
Character Stored
↓
Main Program Continues
```

The CPU only processes serial data when necessary.

Advantages of Interrupts

Interrupt-driven communication offers several important advantages.

- The CPU is free to perform other tasks.
- Incoming characters are less likely to be lost.
- Long calculations do not interfere with serial reception.
- Multiple peripherals can operate simultaneously.

For these reasons, interrupt-driven USART communication is widely used in professional embedded systems.

Using Interrupts with the Library

One of the goals of the MikroDuino USART library is to hide the complexity of configuring the AVR interrupt registers.

Instead of manually enabling

- RX Complete Interrupt
- Data Register Empty Interrupt
- Transmit Complete Interrupt

the library provides a clean interface for enabling interrupt-driven communication. Once enabled, the library automatically invokes your callback or updates its internal receive buffer whenever a character is received.

This allows the application to process complete messages without constantly polling the hardware.

Polling vs. Interrupts

Both techniques have their place.

Polling	Interrupts
Easier to understand	More efficient
Good for simple projects	Better for multitasking
Small programs	Complex applications

Polling

Beginner friendly

Interrupts

Professional embedded systems

For learning USART programming, polling is an excellent starting point.

As your applications become larger, interrupt-driven communication allows the microcontroller to respond quickly to incoming data while continuing to perform other tasks.

Interrupt-Driven Serial Communication

"Let the Hardware Do the Work"

Chapter Introduction

Until now, every USART example in this book has used **polling**.

The program continuously checks whether the USART is ready to transmit or whether a character has been received.

For example,

```
if (USART0.available())
{
    char ch = USART0.read();
}
```

Although this technique is simple and easy to understand, it has an important limitation.

Imagine a robot that must

- receive serial commands,
- read sensors,
- update an LCD,
- control motors,
- blink status LEDs,

all at the same time.

If the program spends most of its time asking

"Has a character arrived?"

then valuable CPU time is wasted. Most of the time, nothing has happened. The microcontroller repeatedly checks the USART only to discover that no new data has arrived. This approach works well for simple applications, but larger embedded systems require something more efficient.

Let the USART Interrupt the CPU

Instead of continuously asking whether data has arrived, imagine allowing the USART to notify the CPU only when something actually happens.

This is exactly what interrupts do. Think of polling as repeatedly walking to your mailbox every few minutes to see whether a letter has arrived.

```
Walk to mailbox
```

```
↓
```

```
No mail
```

```
↓
```

```
Walk again
```

```
↓
```

```
No mail
```

```
↓
```

```
Walk again
```

```
↓
```

```
No mail
```

Most of these trips are unnecessary.

Now imagine your mailbox rings a bell whenever new mail is delivered.

```
Work normally
```

```
↓
```

```
Bell rings
```

```
↓
```

```
Collect mail
```

```
↓
```

```
Continue working
```

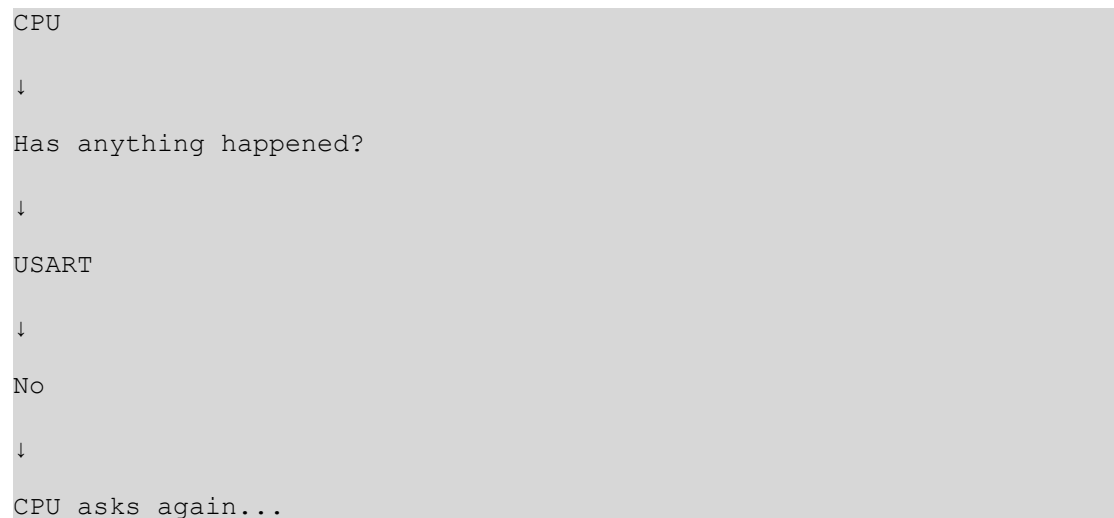
You only stop working when there is actually something to do.

The AVR USART works exactly the same way. Instead of constantly checking the USART, the CPU continues executing the main program. Whenever the USART needs attention, it interrupts the processor automatically.

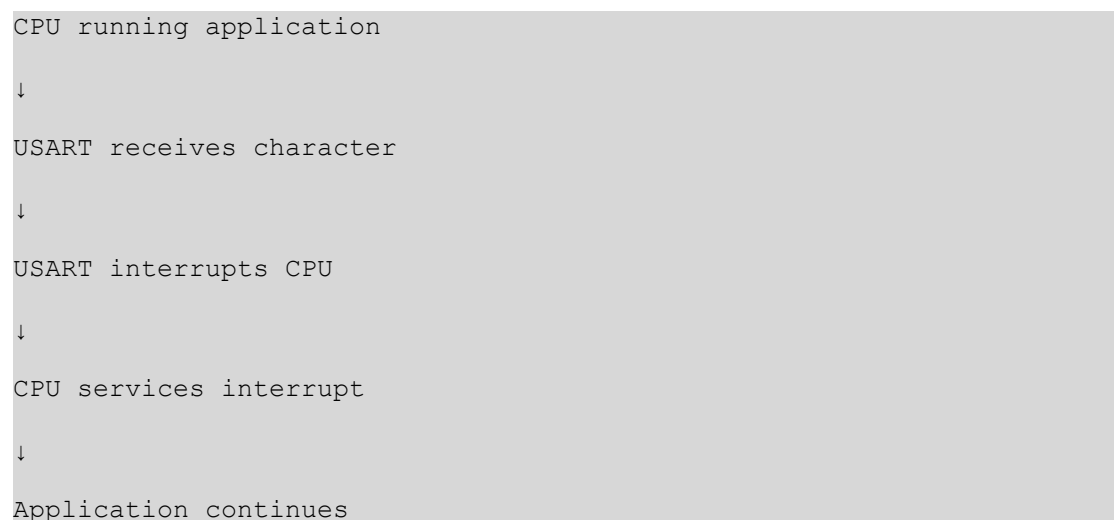
Polling versus Interrupts

Both techniques accomplish exactly the same task. The difference is **who starts the conversation**.

Polling



Interrupts



Notice that during interrupt-driven communication the CPU spends almost no time checking the USART.

Why Interrupts Matter

Interrupts make programs more responsive.

Consider this program.

```
while(true)
{
    readSensors();

    updateDisplay();

    calculatePID();

    controlMotors();

    if(USART0.available())
    {
        processCharacter();
    }
}
```

What happens if

- `calculatePID()` takes several milliseconds?
- Another character arrives during that time?

If characters arrive faster than they are read, the USART receive buffer can overflow. With interrupt-driven reception, the received byte is captured immediately, even while the CPU is busy elsewhere.

This greatly improves reliability.

Three USART Interrupts

The AVR USART provides three interrupt sources.

Each serves a different purpose.

Interrupt	Trigger	Typical Use
RX Complete	A character has been received	Receive data
Data Register Empty (UDRE)	Ready for another transmit byte	Buffered transmission
TX Complete	Entire frame transmitted	RS-485 direction control, transmission complete notification

Together these interrupts allow the USART to communicate almost entirely without polling.

RX Complete Interrupt

This is the interrupt most programmers use first.

Whenever a character arrives,

```
RX Pin
↓
Shift Register
↓
UDR0
↓
RXC0 Flag
↓
RX Complete Interrupt
```

Instead of waiting inside

```
if (USART0.available())
```

the hardware immediately calls an interrupt service routine.

The received byte can then be copied into a software buffer before another character arrives.

Data Register Empty Interrupt

Receiving data is only half of serial communication. Transmission can also be interrupt driven. Suppose your program wants to send

```
HELLO WORLD
```

Polling requires

```
Wait
↓
Load UDR0
↓
Wait
```

```
↓  
Load UDR0  
↓  
Wait...
```

The CPU repeatedly waits for the transmit register. The UDRE interrupt removes this waiting.

Whenever the USART becomes ready for another byte, the interrupt automatically loads the next character from a transmit buffer. The CPU performs no polling whatsoever.

This is the foundation of **non-blocking serial communication**.

TX Complete Interrupt

The final USART interrupt occurs after the last stop bit has completely left the transmitter. Many beginners assume this is identical to UDRE.

It is not. UDRE means "You may load another byte."

TX Complete means "The last bit has physically left the TX pin."

This distinction is extremely important when working with RS-485 networks, where the transmitter must not be disabled until the final stop bit has been transmitted.

Software Buffers

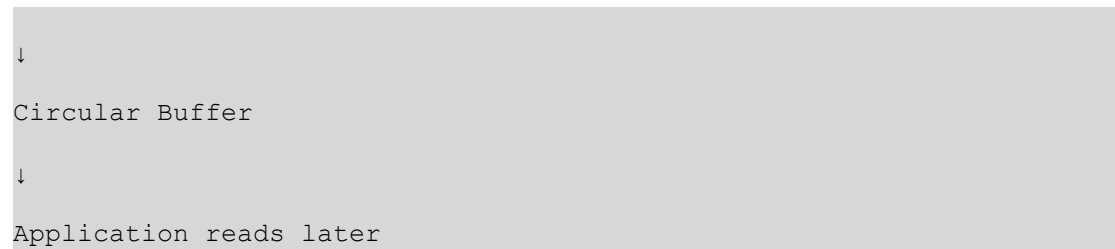
Interrupts introduce another challenge. Suppose five characters arrive almost simultaneously.

```
A  
B  
C  
D  
E
```

The application cannot process them instantly.

Instead, the interrupt stores them inside a software receive buffer.

```
Receive Interrupt
```

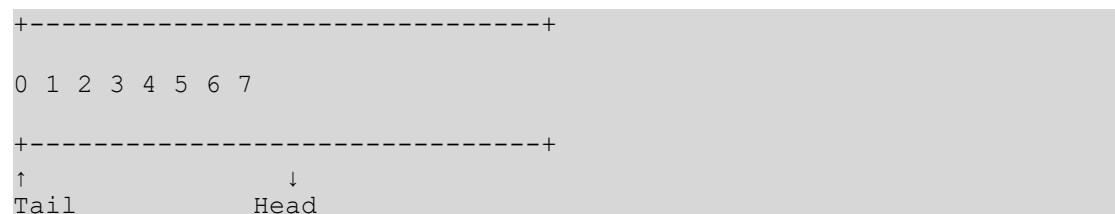



This allows the main program to retrieve characters whenever convenient. Without such a buffer, incoming data could easily be lost.

Circular Buffers

A circular buffer is one of the most useful data structures in embedded programming.

Unlike a normal array, it behaves like a ring.



When the head reaches the end of the array, it simply wraps around to the beginning. No data needs to be moved. This makes circular buffers extremely fast and memory efficient.

Professional embedded systems use circular buffers extensively for

- USART
- SPI
- I²C
- CAN
- USB
- Ethernet

MikroDuino SDK already implements this internally, allowing applications to receive and transmit data continuously with minimal CPU overhead.